



Aristotle University of Thessaloniki
School of Informatics

Distributed Computation Engine

Μηχανή Κατανεμημένου Υπολογισμού

Thodoros Kaimasidis

Student ID: 193

Supervisor: Petros Nicopolitidis

September 24, 2026

Prologue

The question behind this thesis is an old personal one: how can two slow machines be combined to run something faster? Many PC enthusiasts ask it, and data centres and distributed systems have since answered it at industrial scale. I chose this topic because answering it properly means passing through every layer of a computing system: the process and the C runtime, the Linux kernel and its network stack, containers and their orchestration, and the algorithms of distributed systems. Few professional settings offer the chance to work across all of these at once.

The benefit has been exactly that exposure. Building the engine forced decisions about what to reuse and what to write from scratch, and showed in practice what is well known in theory: running threads in parallel is easy, starting multiple processes is easy, and the difficulty lies in communication, exclusivity, and the bottlenecks they create.

Abstract

This work is trying to solve a fundamental issue, that of combining many computing systems in order to create one of larger capacity, without however creating something that is embarrassingly-parallel. The exposure to the underlying mechanisms of the systems involved is also a welcome challenge, and with this I want to distill experience that would be hard to amass in a professional setting with the exciting field of research in the area of massive computation. Inspiration is drawn from the distributed systems applicable to commercial applications and software, as well as the more scientific endpoints of linux and the fundamental technologies used. The result resembles a RPC framework, with reusable and extensible components that for the most part bond well to the underlying structure of the orchestrator that manages the lifecycles of the userspace programs we are going to be using to have run the set of computations, of which results we are going to base ourselves on for the evaluation.

Περίληψη

Μηχανή Κατανεμημένου Υπολογισμού
Θόδωρος Καϊμασίδης

Η παρούσα εργασία επιχειρεί να αντιμετωπίσει ένα θεμελιώδες πρόβλημα: τον συνδυασμό πολλών υπολογιστικών συστημάτων ώστε να προκύψει ένα σύστημα μεγαλύτερης υπολογιστικής ισχύος, χωρίς όμως αυτό να περιορίζεται σε προβλήματα που παραλληλοποιούνται τετριμμένα (εμβαρρασινγκ parallel). Η επαφή με τους υποκείμενους μηχανισμούς των συστημάτων που εμπλέκονται αποτελεί επίσης μια ευπρόσδεκτη πρόκληση, μέσα από την οποία επιδιώκεται η απόκτηση εμπειρίας που θα ήταν δύσκολο να συγκεντρωθεί σε επαγγελματικό περιβάλλον, στο ενδιαφέρον ερευνητικό πεδίο του υπολογισμού μεγάλης κλίμακας. Η έμπνευση αντλείται από τα κατανεμημένα συστήματα που χρησιμοποιούνται σε εμπορικές εφαρμογές και λογισμικό, καθώς και από τις πιο επιστημονικές πτυχές του Λινυξ και των θεμελιωδών τεχνολογιών που χρησιμοποιούνται. Το αποτέλεσμα προσομοιάζει ένα πλαίσιο ΡΠ, με επαναχρησιμοποιήσιμα και επεκτάσιμα δομικά στοιχεία που, στο μεγαλύτερο μέρος τους, ταιριάζουν στη δομή του ενορχηστρωτή ο οποίος διαχειρίζεται τον κύκλο ζωής των προγραμμάτων χώρου χρήστη που εκτελούν τους υπολογισμούς, στα αποτελέσματα των οποίων βασίζεται η αξιολόγηση.

Contents

Abbreviations

ABI	Application Binary Interface
AF_XDP	Address Family eXpress Data Path
API	Application Programming Interface
CPU	Central Processing Unit
CRI	Container Runtime Interface
CRIU	Checkpoint/Restore In Userspace
CSI	Container Storage Interface
DAG	Directed Acyclic Graph
DHT	Distributed Hash Table
DNS	Domain Name System
DPDK	Data Plane Development Kit
eBPF	extended Berkeley Packet Filter
FNV	Fowler–Noll–Vo hash
HPA	Horizontal Pod Autoscaler
HTTP	Hypertext Transfer Protocol
IPC	Inter-Process Communication
JIT	Just-In-Time compilation
JVM	Java Virtual Machine
MPI	Message Passing Interface
mTLS	mutual Transport Layer Security
NIC	Network Interface Controller
OCI	Open Container Initiative
PID	Process Identifier
PVC	PersistentVolumeClaim
RBAC	Role-Based Access Control
RDMA	Remote Direct Memory Access
RPC	Remote Procedure Call
TCP	Transmission Control Protocol
TLS	Transport Layer Security
UDP	User Datagram Protocol
UMEM	AF_XDP user memory region
VM	Virtual Machine
XDP	eXpress Data Path

Chapter 1

Introduction

This thesis is about computation, systems, distributed computation and their intertwining in an interesting project. The need is accelerated computation via the means of one or more computers/computation nodes, which may have lost some of its importance in our days, thanks to many available multi-core consumer and server computer processors, but it's one aspect of computing systems that's extremely rewarding and has too much to teach in the process of its implementation. The problem visited is a personal question of mine from when I was way younger, how can I combine two slow machines to run something faster.

1.1 How can we combine two or more computers to accelerate computation of one or multiple tasks?

This is something that many pc enthusiasts dream of doing, combining the power of multiple cpus or computers for more compute, and it's actually been a huge trend with the advent of data centers and distributed systems evolution and adoption on the business of software development and deployment. The case reviewed here is one about normal pcs with clear boundaries and store bought interconnects (Ethernet) facilitating their intercommunication. The first challenge met is the design of the system, unlike the 'system-design' paradigm of today's development circles however the architecture here can scale up and down, traverse and live in different layers. For example computation is discussed in the context of an os, also in the process context of the C runtime and also with the linux operating system being open source we have access to both with one tool: the C programming language. Further, concurrency is a "solved" issue, the problem is the communication and mutual exclusion of what constitutes a thread in our application, it's been a struggle designing something that is not a mere "embarrassing-parallel" machine, while it's been a similar note for the application of the distributed systems side of algorithms for consensus and communication between nodes.

1.2 The hard part

The hard part comes before the implementation, the decision of what to focus on, what to work on, what to reuse and what to write ourselves. The result of this work is software that can run on two or more machines and via a gateway obfuscate the raw computing power of both to provide higher compute than each of the constituents, hopefully in a scalable way (and in a scalable way that makes sense). For a start we need to define the "computation", what exactly is going to be worked up on: the choice has been made for this to be a set of input data and a function that's to run on them. This comes instead of a process, which in linux is completely open to a developer to see, even if there needs to be a kernel loadable module. Furthermore, this pair constitutes a "job", and the objective is to be able to distribute them in the environments that we control. A

plain worker written in C or Java for example would not suffice here, it's clear that there is a need of a head and a worker component on each installation. As a result after the choice of the execution mechanism, what is needed is a routing algorithm, or a choice of a routing strategy in order to be able to provide an actual higher capacity for computations across two systems, which in our finalized project is taken care of the head (written in C) that constitutes a distributed system. And finally, the landing ground for these has been set to be the kubernetes orchestrator engine, which means packaging in containers and managing of the compute nodes accordingly, this choice has been made because the philosophy of this technology perfectly contributes to the idea of the pooling together of the resources of multiple computers and also is in and of itself an interesting endeavor since much of the modern infrastructure operates in the form of a kubernetes cluster. What has been clear from the start of the project however is the same as what struck me most when I first studied concurrency, running threads in parallel is easy, initiating multiple processes is also easy, problems arise when we have exclusivity and in this way bottlenecks.

1.3 Structure of this Thesis

This thesis puts into context the software project that is the showcase of the work. In total it explains the technologies used, explores the underlying concepts and the mechanisms that are exposed via apis or the ones that are applicable to the project in a meaningful way. A more general note of exploration and care has been given because the whole project is a reflection of my own personal interests.

Along the way we also touch on the mechanisms that linux has to handle execution and process creation and management, what a process is, what it sees and what its universe comprises of, and together with the syscalls and C these set the foundations of the project.

Docker and Kubernetes are covered in Chapter ??, along with mentions of collaborating projects such as CRIU and Karpenter, facilitating process migration and “compute node” discovery.

In Chapter ?? XDP is covered, a special project that's part of the eBPF family of programs that's seen recent popularity and applicability on a huge number of projects both commercial and research focused, together with gRPC, the communication framework used for the control plane.

Chapter ?? then discusses Protocol Buffers, along with the overall effect on the performance of the system, how they work and how they can be applied.

Finally, Chapter ?? is the implementation chapter, whose purpose is to give a high level explanation of the architectural decisions made and the routes taken toward the end goal; the major distributed systems algorithms that were considered are mentioned there, along with their place in the project. Chapter ?? closes with the conclusions, the limitations of the current implementation, and the suggestions for its improvement.

1.4 Epilogue

The introduction has set out the question of this thesis, how two or more ordinary computers can be combined to compute faster than one without the problem being embarrassingly parallel, the unit of work chosen to answer it (a job made of data and a function), and the order in which the following chapters build towards the engine.

Chapter 2

Containers, Orchestration, and Scaling

The engine of Chapter ?? is correct but static: a fixed set of worker processes started by hand. To make it scale effortlessly — to add capacity by declaring a number rather than by wiring up machines — its workers must become a fleet that some external system starts, replaces, places, and grows automatically. That system is a container orchestrator [?, ?], and in practice it is Kubernetes running containers built with Docker/OCI tooling. This chapter is a thorough, advanced introduction to both, oriented throughout toward one payoff: run behind an orchestrator, the coordinator/worker engine turns into a *horizontally elastic function service* — tasks submitted, scheduled onto an autoscaled worker pool, and executed on demand. Structurally that is a self-hosted *distributed lambda*, and when tasks form a map/shuffle/reduce DAG it is a *MapReduce*. The recurring theme is a boundary: what the orchestrator may own versus what the engine must never delegate [?].

2.1 Docker and the container model

In one line: a container is a resource-capped, namespaced process — these are enough to simulate an effectively isolated environment.

Historically we could say that containers are the continuation or adaptation of virtual machines, but this is only true in the way that virtual machines are the replacements of actual server machinery. Containers are however the last executable and writable layer on top of an image, in which all the "work" is distilled, and the biggest difference with virtual machines is that containers rely on the host os to run, that is the host kernel, making it easy to have multiple containers in one computer run, and with the isolation mechanisms having been developed them taking turns executing is under complete isolation from each other. Images need to be created by dedicated teams collecting all application dependencies, and it needs to be in need only of one kernel in order to run, because all programs that are to run on the container will be completely separated from the outside world. Otherwise it is an ordinary Linux process (or process tree) started with a restricted view on the host system.

The most important noun is for sure the *image*, which is the read-only basis of the container, just like the java source is the blueprint of a class in which case the container would be an instance of it - an object -, in actuality it would be a running instance of an image with the container being a writable layer on top of it, with what needs to be persisted needing to be attached to a *volume*. The volume (directory) needs to be mount on the container's filesystem.

Relating, the *bind mount*, maps a specific host directory into the container instead, which also facilitates perfectly development needs. Communication to the outside world happens via the docker network drivers.

Containers on Docker's default **bridge** network can reach the outside world, but container-to-container communication does not provide the convenient name-based discovery available on user-defined networks. Consequently, the first practical setup step is almost always to create a

user-defined network, within which containers can refer to one another by name, with Docker providing the necessary DNS resolution.

Docker networks are implemented through different network drivers, each corresponding to a different networking model and use case. The most important drivers are **bridge**, **host**, **none**, **overlay**, and **macvlan**.

The **bridge** driver is the normal choice when several containers run on the same Docker host. Docker creates a virtual network bridge on the host, and containers attached to it receive their own network interfaces and IP addresses. A user-defined bridge is preferable to Docker's automatically created default bridge because it provides automatic DNS-based service discovery and gives the administrator explicit control over which containers belong to the network.

The **host** driver removes much of the isolation of Docker's network. Instead of giving the container its own isolated network stack, the container uses the host's network namespace directly.

The driver **none** provides essentially no network connectivity to the container, which makes it useful for workloads that do not require networking and where network exposure should be minimized.

The driver **overlay** becomes important when containers are distributed across multiple Docker hosts, allowing them to communicate over a shared logical network spanning those hosts.

Finally, **macvlan** takes a different approach: instead of making the container appear as an endpoint behind Docker's virtual networking, it can give the container its own MAC address and make it appear as a device directly connected to the physical network. This is useful for applications that need to participate directly in an existing Layer-2 network.

Configuration enters through environment variables (and, for anything sensitive, files mounted from tmpfs rather than baked into the image, since image layers are readable by anyone who can pull them). In practice these four are created/defined in a Compose file that declares the images, the volumes, the networks and the published ports for a whole set of services in one place, and brings them up and down together, even defining dependencies between containers.

The most important attribute of the container, and the mechanism by which it can be used and considered an analog to a virtual machine is the mechanisms by which we can enforce its isolation: Namespaces restrict what a process can see (PID, mount, network, IPC, UTS, user, and cgroup namespaces, created by clone with the **CLONE_NEW*** flags), cgroupsv2 limit what it can consume (CPU, memory, I/O, and PIDs), seccomp filters restrict which system calls it may make, and dropped capabilities strip privileges otherwise associated with root, which is a main point of podman, which unlike docker, is daemonless and supports rootless containers, allowing a container to run as root internally while that root identity is mapped through a user namespace to an unprivileged host user, thereby avoiding the need for the container runtime to possess host-root privileges, in linux daemons are all root owned. Namespaces are the mechanism employed in kubernetes as well, they create a folder within the isolated processes have visibility off and the outside world is only available through the managing software and the network. These all show that safety is defined in an application context and for that untrusted code needs to run in a VM-based sandbox (gVisor, Firecracker, Kata).

Three tiers. Docker is not one program but three separable concerns, and almost every property that follows — including every security property — depends on which of the three a mechanism belongs to. The *runtime* is the software that actually runs a container: it talks to the kernel, sets the isolation up, and gets out of the way. The *daemon* sits above it, serving build, start and stop requests and owning the lifecycle of the objects a user names — images, containers, volumes, networks. The *orchestrator* sits above that, deciding where containers run, scheduling them, and reacting to failures. The tiers are defined by interfaces rather than by one vendor's product, which is why each is independently replaceable.

What the runtime builds. A container is not a virtual machine. It is an ordinary Linux process (or process tree) started with a restricted view and a bounded appetite, built entirely from

Linux kernel primitives. Two mechanisms *construct* it: *namespaces* restrict what it can see — PID, mount, network, IPC, UTS, user, cgroup and time, created by `clone` with the `CLONE_NEW*` flags — and *cgroups v2* bound what it can consume (CPU, memory, I/O, PIDs). Three more *constrain* it once built: dropped *capabilities* strip root’s powers one by one, a *seccomp* filter allows only the syscalls it actually needs, and a mandatory access control profile (AppArmor’s path-based rules, or SELinux’s contexts) confines it at the kernel’s own boundary. The distinction is worth holding on to: the first two make the container, the last three make it safe. There is no guest kernel; a container shares the host’s, which is why it starts in milliseconds and also why, by default, it is an isolation boundary but not a security boundary against a kernel exploit — untrusted code wants a VM-based sandbox (gVisor, Firecracker, Kata).

The runtime’s division of labour. The Docker CLI talks to a daemon that delegates to `containerd`, which starts a *shim* per container, which invokes `runc`. `runc` sets up the namespaces, cgroups and pivot-root, installs the seccomp filter and capability set, and `execves` the entrypoint per the OCI runtime spec — and then exits, leaving nothing of itself behind. The shim is what remains: it becomes the container’s parent, holds its `STDIN/STDOUT` open and reaps it, which is why a container survives a restart of the daemon above it. Because the interface is the runtime spec, the runtime is replaceable (`crun`, `youki`, or a VM-backed implementation), and Kubernetes talks to the same lower layers through the CRI (Container Runtime Interface), so Docker-the-daemon is not required on a cluster — `containerd` suffices.

What the daemon manages: images. Images package the filesystem and metadata, and stand to containers as a class stands to an object. An OCI image is an ordered set of tarball *layers* unioned at runtime (overlays) into a root filesystem, plus a JSON config (entrypoint, env, exposed ports). The daemon does the stacking; what the container sees is a single tree, with one thin writable layer on top that dies with it. Layers are content-addressed, so identical layers are cached and shared across images and pulled once. The practical discipline: order the Dockerfile so rarely-changing layers (dependencies) come before volatile ones (your binary), use *multi-stage* builds to keep compilers out of the runtime image, and ship a *distroless* or `scratch` base — for the statically linked C worker of this project, a few megabytes with almost no attack surface. Reference images by *digest* (`image@sha256:...`) in production, never by mutable tag, so a redeploy cannot silently change the binary.

What the daemon manages: networks and volumes. The same tier owns the two objects that outlive a container. Each container receives its own network namespace and a virtual interface whose far end sits on a host bridge, with routing between them and NAT outward; a published port is a rule mapping a host port inward, and it is the one place the isolation is deliberately opened. That default model puts every container on one L2 segment, which is the structural reason inter-container traffic needs separating rather than trusting. Persistent state arrives by mount: a named volume managed by the daemon, a bind mount of a host path, or `tmpfs` for secrets that should never reach a layer. None of it is distributed — two cells on two hosts sharing a volume means a network filesystem, with all of its consistency behaviour intact.

PID 1 and shutdown. A container’s entrypoint runs as PID 1 in its PID namespace, which changes signal semantics: the kernel delivers no default action for signals to PID 1, and PID 1 must reap orphaned children. So the entrypoint must be the real process (via `exec`), or a tiny `init`, or `SIGTERM` is ignored and the orchestrator’s graceful-stop request is dropped. Correct `SIGTERM` handling — stop accepting work, finish or hand back in-flight tasks, exit — is what turns a worker eviction from a lease-expiry storm into a clean handover, and it is the single highest-value piece of container-awareness the engine needs. Resource limits are cgroup limits: a memory limit yields a deterministic cgroup-OOM kill (a clean, attributable crash), while a CPU limit *throttles* — descheduling the process for the rest of the period — which stalls heartbeats and is the most under-diagnosed cause of false failure detection (watch `cpu.stat`’s `nr_throttled`).

The third tier. Everything above is one host. The orchestrator is the tier that decides which host, keeps the declared number of instances running, and replaces the ones that fail —

and because images, networks and volumes are already named, portable objects, it can do so by recreating rather than by moving. That is the subject of the next section, and it is also the boundary of what the container model offers this thesis: relocation at the boundary, not migration of a running computation.

2.2 Kubernetes: orchestration

Mainly kubernetes is an orchestrator for deploying applications, and is generally the de facto infrastructure for distributed systems built today. Kubernetes is a software to manage and deploy applications, literally an orchestrator, the end product is a cluster that is running inside a network of vms, containers, servers. Primarily comprised of two entities (and their constituents): the control-plane and the nodes (data-plane), the management happens declaratively, the desired state is described, and the control is tasked with evaluating the current state, when something is diverging actions are taken to reconcile that. Usually, user-space code will be packaged as a container, and a container will be wrapped in a pod (the smallest deployable unit), next higher-level controllers are going to assist their function across the cores or machines allocated and specified for operation. Although one mostly resorts to distributed architectures for performance gains and reasons, Kubernetes as a manager prides itself in being scalable, reliable and fault tolerant, which are actually the most important aspects for a reliable system, especially with the availability standards that are at play today. What you are buying when you adopt k8s however is efficiency, velocity in deployments, scaling and an abstraction of your infrastructure, which is the opposite of technical debt, as Kelsey Hightower notes in his book. Immutability in the context of Kubernetes is the strategic decision of not updating resources in a way that is not explicitly stated and baked in the configuration or creation logic, instances of an application need to be exactly the same, but it's not possible reliably without an automation layer or the separation of development and deployment that we are able to achieve with docker, Kubernetes and the related technologies. The developer needs to provide solutions that are going to be turned on and off once, they should work with no more than their base arguments and environment, for example updating a library could equally well be achieved by logging in the container and installing the new dependency, or by creating a new one with the updated library baked in, kill the old and launch the new one. Declarative configuration is also easier to understand since it's an 'image' of the world it wants to create, rather than an iteration on a starting point. Kubernetes owned apis and objects are the pods, or group thereof, the actual deployable units that are collections of containers, then services like load balancing, naming and discovery, namespaces that provide isolation the same way docker does for containers and ingress objects that constitute a frontend towards multiple microservices. All these have the effect of isolating the machine from the userspace application that's running, having a negating effect on costs.

Below are the most important control plane components explained:

2.2.1 kubectl / kubeapiserver

What it is: The front door and brain of the control plane. It exposes the Kubernetes API. This is a command-line tool that accesses the kubernetes api, kubectl is just the official image of the client. Typical commands can provide information for the components of a simple cluster like the controller-manager, the scheduler, finally etcd, which is the storage of the cluster.

- **The Only Direct Interface to etcd:** kube-apiserver is the only component in the entire cluster that communicates directly with etcd, while every other control plane component (scheduler, controllers) and worker component (kubelet) has to go through the API server. That means that this is not the writer and that after etcd writes the info and metadata about the state it's only readable in one place, this comes with the extremely important advantage of

providing one place to enforce concurrency control for readers and writers on the distributed system that's to be managed.

- **Stateless & Scalable:** Because `kube-apiserver` stores no local state (it delegates all state persistence to etcd), you can easily scale it horizontally behind a load balancer for High Availability (HA) - installing more instances of it -.
- **Request Pipeline:** Every request, whether issued via `kubectl` or code, moves through a strict processing pipeline before it is ever persisted, and it takes the form of a verb such as `get`, `list`, `watch`, `create`, `update`, `patch`, or `delete` applied to a resource (or a subresource like `pods/exec` or `scale`).
 1. **Authentication:** Confirms the identity of the caller, using mechanisms like client certificates, bearer tokens, or an external identity provider.
 2. **Authorization:** Checks whether that identity is permitted to perform the requested action, typically enforced through Role-Based Access Control (RBAC).
 3. **Admission Control:** Runs a chain of controllers that can mutate or validate the object on its way in, for example injecting default resource limits or blocking requests that violate security policy.
 4. **Storage:** Finally, the API server writes the resulting state to etcd, persisting the state.

`kubectl` is the primary tool for interacting with Kubernetes objects, which are organized into namespaces (folders you can target with `-namespace`, or view all of at once with `-all-namespaces`) and accessed through contexts stored in `~/.kube/config`, which let you set a default namespace and cluster/user combo via `kubectl config set-context` and `use-context`. To view objects, `kubectl get` lists resources (optionally with `-o wide`, `-o json`, `-o yaml`, or JSON-Path queries via `-o jsonpath` for extracting specific fields), while `kubectl describe` gives a fuller human-readable rundown including related events. Objects themselves are defined as YAML or JSON files and managed declaratively with `kubectl apply -f`, which creates or updates only what's changed (previewable with `-dry-run`) and keeps a history of prior configurations you can inspect with `view-last-applied`; you can also edit objects interactively with `kubectl edit`, or remove them with `kubectl delete`, which executes immediately without confirmation. Finally, labels and annotations — arbitrary tags on objects — are managed with `kubectl label` and `kubectl annotate` (requiring `-overwrite` to replace an existing label, and a trailing `-` on the label name to remove one).

2.2.2 etcd

What it is: A strongly consistent, distributed key-value store based on the Raft consensus algorithm, where functions as the storage of all of the api objects.

- **Cluster Storage of state:** Every single object created in Kubernetes (Pods, Services, CRDs, ConfigMaps) lives inside etcd.
- **Watch Mechanism:** Instead of components constantly polling the database (which degrades performance), etcd supports streaming updates. The API server watches etcd and streams real-time events to the controllers/schedulers.
- **Consistency Over Availability:** etcd prioritizes data consistency over partition tolerance, guarding from 'split-brain' conflicts.

2.2.3 kube-scheduler

What it is: The control plane process responsible for deciding which worker node each newly created Pod should run on, kind of like a scheduler for an OS, but without the preemption.

- **Decoupled Decision Maker:** The scheduler never actually starts a container or pulls an image itself; that responsibility belongs entirely to the kubelet running on the chosen node, which is responsible for 'touching' the pods. It is not able to allocate a position to a node that has just entered the cluster, if we want dynamicity we should use an 'extension' of Kubernetes like the Karpenter or the 'cluster-autoscaler'.
- **Two-Phase Placement Engine:** Placement happens in two distinct passes over the available nodes.
 - **Filtering (Predicates):** The scheduler first eliminates any node that simply cannot host the Pod, for reasons such as insufficient CPU or memory, an incompatible taint, or a missing required node label.
 - **Scoring (Priorities):** Among the nodes that remain, it then ranks each one by fit — for example how evenly workloads are spread across zones, proximity to needed data, or overall resource efficiency — and assigns the Pod to whichever node scores highest. Mainly used for reallocating pods after failures or on scaling.

2.2.4 Cluster Autoscaler / Karpenter

What it is: A component that runs *alongside* the control plane, rather than as one of its core pieces, whose job is to add or remove worker nodes based on demand, not part of the default installation of Kubernetes.

- **The Trigger:** Both tools watch the API server for Pods stuck in the **Pending** state because no current node can satisfy their resource requests, and treat that backlog as a signal that more capacity is needed.
- **Cluster Autoscaler:** The older, more established of the two. It works in terms of pre-defined node groups (an AWS Auto Scaling Group, a GCP Managed Instance Group, etc.) and simply scales those groups up or down, meaning new nodes come in whatever instance size and shape the group was already configured with.
- **Karpenter:** A newer, more flexible alternative that provisions individual nodes directly against the cloud provider's API, without needing pre-defined node groups. It picks the specific instance type, size, and even spot-vs-on-demand mix that best fits the exact Pods waiting to be scheduled, which typically means faster and more tightly bin-packed provisioning.
- **Scaling Down:** Both tools also work in reverse, continuously looking for nodes that are underutilized or empty and safely draining and terminating them to avoid paying for idle capacity.
- **Division of Labor:** Once a new node registers with the API server and is marked **Ready** (via the Node Controller in `cloud-controller-manager`), it becomes just another candidate node, and it is **kube-scheduler** — not the autoscaler — that decides which of the pending Pods actually land on it.

Finally it should be noted that the constituents of a cluster are in and of themselves deployed via Kubernetes, Kubernetes DNS provides naming and discovery for the services defined, also a replicated service, additionally there is the Kubernetes proxy that's responsible for load balancing and traffic routing over the network of the cluster.

The most important moving parts are the data plane nodes:

2.2.5 Worker Node

What it is: Every worker node runs the same three-piece stack: an agent, a runtime, and a network proxy.

```
Worker Node
|-- kubelet
|-- Container Runtime
|-- kube-proxy
```

2.2.6 kubelet

What it is: The main Kubernetes agent running on every worker node.

- Communicates with the `kube-apiserver` to learn what should be running on its node.
- Receives Pod specifications describing the containers, images, and resources a Pod needs.
- Ensures the assigned Pods are running by starting their containers via the container runtime.
- Monitors containers to confirm they stay in the expected state.
- Reports node and Pod status back to the control plane so it always has an accurate picture.
- Performs health checks and restarts containers when necessary, enabling self-healing.

2.2.7 Container Runtime

What it is: The container runtime is responsible for actually running the containers.

Examples include:

- `containerd` – the most widely used runtime, and the default in most managed Kubernetes offerings.
- `CRI-O` – a lightweight runtime built specifically for Kubernetes.

The kubelet communicates with the runtime through the Container Runtime Interface (CRI), so any compliant runtime can be swapped in without changing kubelet itself.

2.2.8 kube-proxy

What it is: `kube-proxy` provides the networking functionality required for Kubernetes Services.

- Maintains network rules on the node, typically using `iptables` or `IPVS`.
- Enables communication with Services by resolving their stable virtual IP to real Pod addresses.
- Routes traffic to the appropriate Pods, load-balancing across all healthy backends.
- Implements Service types such as `ClusterIP` and `NodePort`, giving each a working network behavior.

What it is: Every kind of Kubernetes object shares the same `apiVersion/kind/metadata/spec/status` shape and the same declarative contract — you write `spec`, Kubernetes writes `status`, and a controller continuously reconciles the two. What differs from one kind to the next is what that controller actually *does*: a Pod just runs containers, a Deployment manages rolling updates, a Job runs something to completion. The rest of this section is a reference card for each of the kinds you'll use most, showing how they relate to one another, what operations are available on each, and a minimal YAML example.

2.2.9 How the Kinds Relate

- **Deployment** owns a **ReplicaSet**, which owns **Pods**.
- **CronJob** owns a **Job**, which owns **Pods**.
- **DaemonSet** owns **Pods** (one per node).
- **StatefulSet** owns **Pods** (stable identity and storage).
- **Service** selects **Pods** — it does not own them.
- **ConfigMap** and **Secret** are mounted into **Pods** — again, no ownership.

Solid arrow = owns / creates (garbage-collected together via **ownerReferences**). **Dashed arrow** = selects or mounts into (no ownership, no cascading delete).

2.2.10 Pod

What it is: The smallest deployable unit in Kubernetes — one or more containers that share a network namespace and storage volumes, and are always scheduled together on the same node. Pods are described in a manifest, that describes them to kubernetes since they are the core object, pods can consist of multiple containers or can host one and communicate over the network, each one is controlled by a cgroup for resource limits and will share the appropriate namespaces with neighboring , not identical pods. Most important design decision for splitting the application's components into pods is what scaling needs are there for each, since a pod is the equivalent of an atom for kubernetes. Kubectl will accept and store teh manifests in etcd, then the scheduler will take over, and will ensure if possible to distribute the pods onto different machines for reliability.

You can:

- **Create** — `kubectl apply -f pod.yaml → POST /api/v1/namespaces/{ns}/pods`.
- **Inspect** — `kubectl get/describe pod → GET` the Pod resource or its **status** subresource.
- **Exec into it** — `kubectl exec → POST .../pods/{name}/exec`.
- **Read logs** — `kubectl logs → GET .../pods/{name}/log`.
- **Port-forward** — `kubectl port-forward → POST .../pods/{name}/portforward`.
- **Kill it** — `kubectl delete pod → DELETE .../pods/{name}`. If it's owned by a **ReplicaSet**, **DaemonSet**, or **StatefulSet**, a replacement appears automatically.
- **Cannot** scale, roll back, or migrate itself — Pods are disposable by design; those behaviors belong to the controllers above it. Kubectl exposes a basic api that's similar to docker for inspecting pods and scheduling them, that's because they can be almost identical in content.

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  labels:
    app: nginx
spec:
  containers:
  - name: nginx
    image: nginx:1.27
    ports:
    - containerPort: 80
```

2.2.11 ReplicaSet

What it is: A controller that guarantees a fixed number of identical Pods are running at all times, matching Pods purely by label selector rather than by name. These provide redundancy, scaling and sharding, and forms the basis for the reliability and fault tolerance of Kubernetes. It's designed to group pods together and expose them as one, management takes place via reconciliation loops. This is the first instance of the configuration of the desired state defined earlier. What's being stated is the number of pods and their configurations, mechanisms have been defined to 'reconcile' each and every discrepancy from the blueprint. Labels are used to mount a pod to a replica set, also replicas are not coupled with the services mentioned later in this thesis about service discovery, dns and load balancing because it goes against the loose coupling principles of Kubernetes.

You can:

- **Create** — `kubectl apply -f rs.yaml → POST .../replicasets`.
- **Scale** — `kubectl scale rs -replicas=N → PATCH .../replicasets/{name}/scale`.
- **Delete** — `kubectl delete rs → cascades to its owned Pods via ownerReferences`.
- **In practice** you rarely create these directly — a Deployment creates and manages them for you.

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: nginx-rs
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
```

2.2.12 Deployment

What it is: The standard controller for stateless applications, managing ReplicaSets on your behalf so you get rolling updates, rollbacks, and self-healing without touching Pods directly.

You can:

- **Run / create** — `kubectl create deployment or apply -f deploy.yaml → POST .../deployments`.
- **Scale** — `kubectl scale deployment -replicas=N → PATCH .../deployments/{name}/scale`.
- **Update (rolling deploy)** — `kubectl set image deployment/x c=img:v2 → PATCH spec.template`; a new ReplicaSet is created and scaled up while the old one scales down.
- **Roll back** — `kubectl rollout undo deployment/x → reverts to a previous ReplicaSet revision`.
- **Pause / resume a rollout** — `kubectl rollout pause | resume`.

- **Restart** — `kubectl rollout restart` → patches a pod-template annotation, forcing fresh Pods even with no spec change.
- **Stop / kill** — `kubectl delete deployment` → `DELETE` ...; cascades to its ReplicaSets and their Pods.
- **“Migrate”** — Kubernetes never live-migrates a running Pod. Moving work off a node means cordoning it (`kubectl cordon`) and draining it (`kubectl drain`), which evicts its Pods so the Deployment’s controller reschedules replacements elsewhere.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deploy
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
```

2.2.13 StatefulSet

What it is: Like a Deployment, but for workloads that need a stable, unique network identity and stable per-Pod storage that survives rescheduling — think databases and other clustered stateful apps.

You can:

- **Create / scale / delete** — same commands as a Deployment, but Pods are created, scaled, and terminated in order (`web-0`, `web-1`, `web-2`, ..., reverse order on scale-down).
- **Update** — `spec.updateStrategy` is either `RollingUpdate` (default, ordered) or `OnDelete` (you delete Pods manually to trigger their replacement).
- **Stable identity** — each Pod keeps the same hostname and, via `volumeClaimTemplates`, the same `PersistentVolumeClaim` even after being rescheduled to a different node.
- **Delete** — `kubectl delete statefulset` removes the Pods but, by default, leaves the PVCs (and their data) behind.

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: web
spec:
  serviceName: "web"
  replicas: 3
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
```

```

spec:
  containers:
    - name: web
      image: nginx:1.27
  volumeClaimTemplates:
    - metadata:
        name: www
      spec:
        accessModes: ["ReadWriteOnce"]
        resources:
          requests:
            storage: 1Gi

```

2.2.14 DaemonSet

What it is: A controller that ensures exactly one copy of a Pod runs on every node (or every node matching a selector) — the pattern used for log collectors, node monitoring agents, and network plugins. Typically these are the equivalent of the daemon, background running tasks that are present in oses, and accordingly they have monitoring roles, and this is achieved by replicating one node to each node, installing in this way a daemon.

You can:

- **Create** — `kubectl apply -f ds.yaml → POST .../daemonsets`.
- **“Scale”** — not by replica count; the Pod count tracks the number of matching nodes automatically, so it grows or shrinks as nodes join or leave.
- **Update** — `RollingUpdate` (default) or `OnDelete`, same idea as `StatefulSet`.
- **Delete** — `kubectl delete daemonset` removes the Pod from every node.

```

apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: node-exporter
spec:
  selector:
    matchLabels:
      app: node-exporter
  template:
    metadata:
      labels:
        app: node-exporter
    spec:
      containers:
        - name: node-exporter
          image: prom/node-exporter

```

Hightower states: «ReplicaSets need to be run when the application is completely decoupled from the node you are using, while DaemonSets are for architectures where pods with your whole application must be present on all nodes of the cluster». There is however a way to specify a node subset for specific deployments, labels can be set as selectors for those cases.

2.2.15 Job

What it is: A controller for run-to-completion workloads — it tracks successful completions rather than keeping Pods alive forever, and retries failed Pods up to a limit. A job is the creation of a pod that runs until termination, batch jobs, database migrations and things that need to happen once and completely are the scope.

You can:

- **Run** — `kubectl create job or apply -f job.yaml → POST .../jobs`.
- **Control throughput** — `spec.completions` and `spec.parallelism` decide how many Pods must succeed and how many can run at once.
- **Suspend / resume** — `kubectl patch job x -p '{"spec":{"suspend":true}}'` pauses it without deleting it.
- **Stop / kill** — `kubectl delete job -cascade=foreground|orphan` controls whether its Pods are deleted along with it.

```
apiVersion: batch/v1
kind: Job
metadata:
  name: db-migrate
spec:
  completions: 1
  parallelism: 1
  template:
    spec:
      containers:
      - name: migrate
        image: myapp/migrate
      restartPolicy: Never
```

Once again jobs are managing and deploying pods to get the job done, and in so doing we can achieve parallelization of workloads and even define work queues.

2.2.16 CronJob

What it is: A scheduler that creates a new Job on a cron-style schedule — the Kubernetes equivalent of a crontab entry, but producing real, trackable Job/Pod objects each run.

You can:

- **Create** — `kubectl create cronjob or apply -f cronjob.yaml → POST .../cronjobs`.
- **Suspend** — `kubectl patch cronjob x -p '{"spec":{"suspend":true}}'` stops new runs without deleting the schedule.
- **Control overlap** — `spec.concurrencyPolicy: Allow, Forbid, or Replace`.
- **Trigger manually** — `kubectl create job -from=cronjob/x manual-run-1` runs it immediately, outside the schedule.
- **Delete** — `kubectl delete cronjob` removes the schedule and its recent Jobs.

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: nightly-backup
spec:
  schedule: "0 2 * * *"
  jobTemplate:
    spec:
      template:
        spec:
          containers:
          - name: backup
            image: myapp/backup
          restartPolicy: OnFailure
```

2.2.17 Service

What it is: A stable virtual address that load-balances traffic to whichever Pods currently match its label selector, so clients never need to track individual Pod IPs.

You can:

- **Create** — `kubectl expose deployment/x -port=80` or `apply -f svc.yaml → POST .../services`.
- **Change routing** — editing `spec.selector` or `spec.ports` re-routes live traffic as soon as kube-proxy picks up the updated Endpoints, no Pod restart needed.
- **Choose exposure** — `spec.type`: `ClusterIP` (internal only), `NodePort` (also exposed on every node), or `LoadBalancer` (provisions a cloud load balancer via `cloud-controller-manager`).
- **Delete** — `kubectl delete service` removes the virtual IP/load balancer; it never touches the backing Pods themselves.

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-svc
spec:
  selector:
    app: nginx
  ports:
    - port: 80
      targetPort: 80
  type: ClusterIP
```

Generally, service discovery is a contiguous problem, and abstracts/solves the discovery - literally- of the actual pod that's used to implement the actual service, even a routing table can be used, but in this case specifically we care about dynamicity of this since there are multiple levels of the stack from os to hardware to pod and to container that may be changing. This is why traditional DNS fails, SRV records are an alternative, however going further more suitable to the specific domain and application routing needs to be set up, perhaps with specialized or custom software. For outwards port forwarding there is the nodeport feature that's designed for allowing traffic in one namespace, by being able to reach just one node of the cluster.

2.2.18 ConfigMap & Secret

What it is: Two nearly identical objects for injecting configuration into Pods without baking it into the container image — a ConfigMap for plain values, a Secret for sensitive ones (base64-encoded at rest, and typically further protected by encryption-at-rest and RBAC).

You can:

- **Create** — `kubectl create configmap/secret` or `apply -f → POST .../configmaps` or `.../secrets`.
- **Consume** — mount as a volume (files appear under a directory) or inject as environment variables via `envFrom/valueFrom`.
- **Update** — `kubectl edit` or `apply → PATCH ...`; volume-mounted files update on the kubelet's next sync automatically, but env-var injections require the Pod to restart to pick up the change.
- **Delete** — `kubectl delete configmap/secret`; Pods already using the old data keep running until they're restarted.

```

apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  LOG_LEVEL: "info"
---
apiVersion: v1
kind: Secret
metadata:
  name: app-secret
type: Opaque
data:
  DB_PASSWORD: cGFzc3dvcmQ= # base64-encoded

```

2.3 The Kubernetes objects and where each fits the engine

This what the context of kubernetes encompasses, defined below are the nouns and verbs of kubernetes, making it clear what actions and objects we can handle for the management of the lifecycle of our userspace programs.

Pod	one+ co-scheduled containers sharing net+IPC ns (localhost, sidecars)	
Deployment	N interchangeable STATELESS pods, rolling updates	-> WORKERS
StatefulSet	stable names/storage, ordered ops	-> COORDINATOR quorum
Job / CronJob	run-to-completion / scheduled batch	-> occasional batch only
DaemonSet	one pod per node	-> node agents
Service	stable virtual IP + L4 LB over a pod set	
Headless Svc	one DNS A-record per pod (no VIP)	-> peer discovery
ConfigMap/Secret	configuration / credentials	
PVC / PV / StorageClass	durable volumes via CSI	-> logs, DB state
Ingress / Gateway	north-south HTTP entry	-> Submit front door
HPA / VPA	autoscale replicas / right-size requests	-> scale workers
PodDisruptionBudget	cap voluntary disruption	-> protect quorum
NetworkPolicy / RBAC / SA / securityContext	least privilege	
CRD + controller	extend the API with your own object + reconcile loop	

Buy consensus; do not autoscale it. A quorum's size is a correctness parameter, not a capacity knob — an HPA must never resize the coordinator StatefulSet, because adding a member is a membership-change protocol (Section ??), not a replica bump. And run *your own* etcd: sharing the cluster's control-plane etcd couples your load to the API server's health, a documented way to break a cluster.

Durability on Kubernetes has its own caveats: a PVC is not a backup (snapshot to object storage and *test restore*); the log's device must give low-latency **fsync**, so prefer local NVMe over network storage whose sync is a network round trip.

2.4 API gateways, ingress, and service mesh

Traffic splits into *north-south* (clients → cluster) and *east-west* (service → service), and each has an edge.

North-south is the Ingress / Gateway API and its controller (Envoy, NGINX, Traefik): one entry point that terminates TLS, routes by host/path, enforces authentication and rate limits, and splits traffic for canary or blue-green rollouts. For the engine this is the *Submit* front door: it authenticates callers, applies per-tenant rate limits and load shedding (Section ??), and routes to the coordinator leader — so the admission and backpressure logic lives at a single, observable choke point rather than being scattered.

East-west is optionally a *service mesh*: a sidecar proxy (Envoy) injected beside every pod that transparently adds mutual TLS (service identity), retries, timeouts, circuit breaking, and per-call telemetry. This is exactly the fault-tolerance and RPC-hardening machinery an RPC layer would otherwise build itself, *externalized* out of the application into the platform. The trade is real overhead (a proxy hop and CPU per pod) and operational complexity, so a mesh earns its place at scale or across trust boundaries, not by default; a C engine that already implements deadlines, retries with budgets, and mTLS in its own RPC layer may reasonably skip it and keep the control it already has.

2.5 Epilogue

This chapter has placed the engine inside the container and orchestration model. A container is an ordinary Linux process whose view is restricted by namespaces and whose consumption is bounded by cgroups, and Kubernetes keeps a declared number of such processes running by reconciling desired against observed state. The boundary drawn throughout is the one the engine depends on: the orchestrator owns the lifecycle of the nodes, while the placement and scheduling of jobs remain the engine's own, and consensus is bought from etcd rather than autoscaled.

Chapter 3

Two Transports: XDP and gRPC

As we see on the implementation of the rudimentary engine, computing costs across machines that are in the same network and not necessarily in close proximity are a detriment to performance.

BPF originally emerged as a technology for efficiently filtering network packets. When you run `tcpdump` with an expression—such as matching a specific host or port—the expression is compiled into optimized BPF bytecode and executed by a sandboxed virtual machine inside the kernel.

Over time, BPF evolved significantly beyond its original purpose of packet filtering. With the introduction of **extended BPF (eBPF)**, the virtual machine gained the ability to execute custom programs in response to many different events occurring within the Linux system. Instead of only examining network packets, eBPF programs can be attached to system calls, kernel functions, tracepoints, network events, and application-level probes.

This makes BPF a powerful mechanism for **observability, tracing, performance analysis, networking, and security**. For example, an eBPF program can monitor system calls, measure how long processes spend waiting for CPU time, trace file-system operations, inspect network activity, or capture stack traces when applications block. These programs run in the kernel while being checked by the BPF verifier, which ensures that they meet safety requirements before they are loaded.

Developers typically interact with BPF through higher-level tools and frameworks such as **BCC** and **bpftool**. BCC is commonly used for building more complex tracing tools and applications, while bpftool provides a simpler interface for writing short tracing programs and one-liners. Together, these tools make it possible to analyze and understand Linux systems without modifying the kernel or adding traditional instrumentation to every application.

Although the technology is formally known as **eBPF** to distinguish it from the original Berkeley Packet Filter, it is now commonly referred to simply as **BPF**. In modern Linux systems, BPF provides a general-purpose way to safely extend kernel functionality and observe system behavior dynamically.

The control-plane/data-plane split that runs through this thesis has a transport at each end, and they pull in opposite directions. The *data plane* — bulk task payloads, results, checkpoint images moving between nodes — wants the lowest possible per-byte and per-packet cost, even at the price of building reliability by hand. The *control plane* — `Submit`, dispatch, status, the worker channel — wants typed, evolvable, reliable request/response with deadlines and streaming, and to fit the Kubernetes ecosystem of Chapter ???. This short chapter presents one technology aimed squarely at each: **XDP** for the hot data path in the kernel, and **gRPC** for the application-layer control path.

3.1 XDP: programmable packet processing at the driver

XDP (eXpress Data Path) is a high-performance networking technology built on top of BPF/eBPF. It allows BPF programs to run at a very early stage of packet processing, immediately after a network packet is received by the network interface and before it passes through most of the Linux networking stack. Because the packet can be processed so early, XDP can make decisions with very low overhead, which makes it particularly useful for high-speed packet processing.

An XDP program can inspect the contents of an incoming packet and decide what should happen to it. For example, it can drop unwanted packets, pass packets to the normal Linux networking stack, redirect packets to another network interface or CPU, or transmit a packet directly back out through the interface. This makes XDP useful for applications such as DDoS protection, firewalls, load balancing, packet filtering, and high-performance networking. XDP (eXpress Data Path) [?,?] runs an *eBPF* program at the earliest point in the receive path — inside the NIC driver, before the kernel allocates an `sk_buff`, and on capable hardware offloaded onto the NIC itself. eBPF is sandboxed bytecode: the kernel *verifier* proves the program terminates and touches only permitted memory, then JITs it to native code, so it runs at kernel speed without the ability to crash the kernel; persistent state lives in *maps* shared with userspace. An XDP program sees each raw frame and returns a verdict: `XDP_DROP` (discard — the basis of line-rate DDoS filtering), `XDP_PASS` (hand up to the normal stack), `XDP_TX` (bounce back out the same NIC), or `XDP_REDIRECT` (send to another NIC, another CPU, or a userspace socket). Because it acts before the sockets/TCP machinery exists, it is where an L4 load balancer (spreading `Submit` or dispatch across coordinators and workers) or a custom router belongs when the per-packet cost of the full stack is the bottleneck. The key idea is that XDP is not a separate replacement for BPF. Instead, it is one of the mechanisms through which BPF programs can be attached to the Linux networking path. While traditional BPF tracing can observe many parts of the system, XDP specifically focuses on processing network packets as early and efficiently as possible.

The piece most relevant to this project’s data plane is **AF_XDP**: a socket type that, paired with an XDP `REDIRECT`, DMA’s frames directly into a userspace memory region (a *UMEM* of fill/completion rings) with *zero copy* and no traversal of the kernel network stack. This is kernel-bypass packet I/O reachable from an ordinary process — the mechanism behind userspace networking frameworks — and it is the natural substrate for a specialized, connectionless bulk transfer of results or checkpoint images between nodes at line rate. The cost is that you inherit everything the stack was doing for you: `AF_XDP` delivers *frames*, not a reliable ordered byte stream, so segmentation, acknowledgement, retransmission, and congestion control are yours to build — which is precisely building an RPC transport from first principles, now at the packet level. Combined with the verifier’s limits on program size and complexity and the need for driver support, this makes XDP/`AF_XDP` a tool for a small number of hot paths — a load-balancing front end, or a bulk data mover — not a general RPC layer.

3.1.1 eBPF, maps, and the verifier

An XDP program is an *eBPF* program: restricted C compiled by `clang` to a sandboxed bytecode, loaded through the `bpf()` syscall, checked by the in-kernel *verifier*, and JIT-compiled to native code. The verifier is what makes running code in the datapath safe, and it shapes how these programs are written: it proves the program halts (loops must be bounded) and that every memory access is in range, so a frame is parsed by a strict *check-then-read* walk — a header pointer is compared against the frame’s `data_end` *before* it is dereferenced — under a small stack and a bounded instruction budget. Persistent and shared state lives only in *maps*: typed key/value tables the program shares with userspace (through `bpf()`) and across CPUs. Four map types carry this project’s datapath: a `BPF_MAP_TYPE_HASH` routing map (`function_id` → cell), an `ARRAY` of per-function counters, and the two redirect targets `CPUMAP` (a pinned core) and

XSKMAP (an AF_XDP socket). The program reaches the kernel only through *helper functions* — `bpf_map_lookup_elem`, `bpf_redirect_map`, and so on — some of which, redirection among them, require the program to declare a GPL-compatible license. If the packet comes from a blocked address, the program can immediately return `XDP_DROP`, preventing the packet from travelling further through the networking stack. If it is legitimate, it can return `XDP_PASS` and allow Linux to continue processing it.

This early execution point is what gives XDP much of its performance advantage: the closer packet processing is moved to the network driver, the less work the system needs to perform before making a decision about the packet.

3.1.2 Attachment modes and the reality of zero copy

The same program can attach in three ways, trading portability for cost. In *offloaded* mode it runs on a capable SmartNIC; in *native* (driver) mode it runs inside the driver’s receive poll, before any `sk_buff` exists — the intended mode, and the only one in which the NIC can DMA a frame straight into userspace memory; in *generic* (SKB) mode it runs later, in the stack after the `sk_buff` is built, which works on any interface (including the `veth` pairs used inside containers) but copies. The distinction is decisive for the zero-copy claim: true zero copy requires native mode *and* an `XDP_ZEROCOPY` socket binding on a driver that supports it, whereas the portable fallback — `XDP_COPY` in generic mode — exercises exactly the same routing logic but pays one copy. Where the benefit is real is therefore a property of the driver and mode, not of the program.

3.1.3 AF_XDP: the UMEM and its four rings

AF_XDP couples the `XDP_REDIRECT` verdict to a userspace socket through a shared memory region, the *UMEM*: a block of process memory, registered with the kernel and carved into fixed-size frames, that in native mode is the NIC’s DMA target so packets land in it with no copy. Four single-producer/single-consumer descriptor rings coordinate ownership. Two belong to the UMEM: **FILL** (userspace hands the kernel empty frames to receive into) and **COMPLETION** (the kernel returns frames whose transmission has finished). Two belong to the socket: **RX** (the kernel hands userspace descriptors of filled frames) and **TX** (userspace hands the kernel frames to send). A frame’s life is thus a cycle `FILL` → (DMA) → `RX` → processing → `FILL`. The bridge from the in-kernel verdict to one specific socket is the `XSKMAP`: userspace registers each socket’s file descriptor at its receive-queue index, and `bpf_redirect_map(&xsks_map, queue_id, ...)` in the program lands the frame in that socket’s `RX` ring. Because the rings are per queue, the design scales by binding *one socket per NIC receive queue* (spread by the hardware’s receive-side scaling) and pinning each to a core, which removes any shared reader and maps one-to-one onto a per-core worker pool; an optional `need_wakeup` flag lets the kernel signal when a `poll()` is actually required, amortizing syscalls at high rates.

3.1.4 How this project uses XDP

In our design XDP is the *routing plane* for the job data path, not a general transport. Each record travels in a UDP frame with a small fixed header (`magic`, `function_id`, `feed_id`, `record_id`, `length`) placed before the struct payload. An XDP program on the receiving node parses *only that header* — never the struct — reads `function_id`, looks it up in an eBPF *routing map* (`function_id` → target cell), and `XDP_REDIRECTs` the frame, either to a `CPUMAP` (a pinned core) or, for zero copy, into the target cell’s **AF_XDP** socket, before the kernel builds an `sk_buff`. The cell’s worker reads frames from its AF_XDP `RX` ring, hands the payload to its thread pool, runs the `dlopen`’d function, and writes results back through the `TX` ring. Two properties make this the right tool here: routing is *format-agnostic* (the payload’s struct/FlatBuffer layout stays owned by the function; XDP touches only the header), and *relocation is a single map write* —

when a cell moves to another node, updating `function_id` $\rightarrow$ `target` makes the stream follow it. The prototype in `experiments/xdp/` implements the parse-and-classify half (per-function counters in an eBPF map, driven by a UDP feeder); the production step replaces the counter with the `XDP_REDIRECT` into `AF_XDP`. Consistent with the RPC-from-first-principles caveat above, we use `XDP/AF_XDP` only on the hot, small-record path and keep ordinary sockets or gRPC for control and moderate streams.

3.1.5 Frame ownership across the pool, and in-kernel admission

Two consequences of the UMEM model matter once frames feed a thread pool rather than a single reader. First, ownership outlives the RX ring: a worker reads its job *in place* from the UMEM frame, so that frame cannot return to the FILL ring until the worker finishes; the reader records each frame’s address alongside the job and recycles it only after `run()` completes, and because several workers recycle into the single-producer FILL ring, those returns are serialized. If every frame is in flight the FILL ring drains and the NIC drops — a natural back-pressure bound set by the frame count and the queue depth. Second, the program can reject work *before* it costs anything: validating the header’s magic and destination in-kernel and returning `XDP_DROP` for non-conforming frames discards traffic aimed at the data-plane port before an `sk_buff`, a wakeup, or an allocation — the same mechanism that makes XDP a line-rate DDoS filter, here serving as admission control for the job stream, while conforming traffic on other ports is passed up untouched with `XDP_PASS`.

3.1.6 Deployment: privileges and containers

Because XDP is a kernel facility and containers share the host kernel, running the router or an `AF_XDP` receiver in a container is a question of privilege and namespace, not of a special runtime. Attaching the program needs `CAP_NET_ADMIN`; loading it and its maps needs `CAP_BPF` (or `CAP_SYS_ADMIN` on kernels before 5.8); the UMEM and maps are locked memory, so `RLIMIT_MEMLOCK` must be raised; and a runtime’s default seccomp policy typically blocks the `bpf()` syscall unless one of those capabilities is present. The program attaches to an interface *in the container’s network namespace*: on a bridge network that is a `veth` endpoint, which supports only generic mode and hence the copy fallback, whereas true zero copy requires a native-XDP NIC in the namespace — host networking, or an SR-IOV virtual function passed in, the arrangement the Kubernetes SR-IOV and `AF_XDP` device plugins provide. This is the same portability-versus-performance boundary as the attachment modes above, drawn now at the container edge.

3.2 gRPC: typed RPC on HTTP/2

gRPC is the opposite choice: a batteries-included, application-layer RPC framework and the de facto standard for service-to-service calls on Kubernetes. Interfaces and messages are declared in *Protocol Buffers*, whose compiler generates client and server stubs and whose tag-based binary encoding gives compact wire size and the backward/forward schema evolution of Section ?? (add optional fields, never reuse a tag). The transport is HTTP/2, which supplies — off the shelf — exactly the mechanisms that chapter built by hand: *multiplexed streams* over one connection, per-stream *flow control*, and header compression. Four call shapes fall out of it: unary, server-streaming, client-streaming, and *bidirectional streaming* — the last being the ideal form for the long-lived worker channel on which the coordinator dispatches tasks and the worker streams heartbeats and completions.

Around this core gRPC provides the production concerns as first-class features: *deadlines* propagated in call metadata (so a client’s deadline bounds all downstream work), cancellation,

status codes carrying retryability, pluggable *interceptors* for authentication, tracing, and metrics, and name resolution with client-side load balancing. In a cluster it slots directly into the Chapter ?? machinery: it traverses Ingress and Gateway controllers, gets mutual-TLS identity and retries from a service mesh for free, and has a standard health-checking protocol probes can use. For this project it is the natural control-plane transport — client→gateway→coordinator, and the coordinator↔worker channel — giving multiplexing, deadlines, streaming, and schema evolution without hand-rolling them. Its costs place it on the control side of the split: HTTP/2 framing and HPACK cost CPU at very high message rates, and because streams share one TCP connection a lost segment head-of-line-blocks all of them (a limitation HTTP/3 over QUIC removes), so gRPC is the wrong tool for the XDP-class bulk data plane.

3.3 Where each fits

	XDP / AF_XDP	gRPC
plane	data (bulk, hot path)	control (API, dispatch)
layer	L2-L4, pre-stack / kernel-bypass	L7 on HTTP/2
unit	raw frames	typed messages + streams
reliability	you build it	provided (over TCP)
evolution	n/a (custom)	protobuf schema, tag-based
k8s fit	CNI dataplanes, L4 LB, offload	Ingress, mesh, mTLS, health
use here	result/checkpoint bulk transfer, in-cluster load balancing	Submit/status, worker channel, inter-component RPC

The two are complementary, not competing: they are the concrete realizations of the same control/data-plane split. gRPC carries the small, typed, reliable control traffic that must be correct and evolvable; XDP/AF_XDP (or, more modestly, plain TCP plus the object store) carries the large, latency-sensitive data traffic that must be fast. Choosing per plane, rather than forcing one transport everywhere, is the same discipline — match the mechanism to the guarantee it must provide — that governs every layer of this system.

3.4 Epilogue

The two transports of this chapter are the two halves of the same control/data-plane split. XDP and AF_XDP give the data plane packet processing before the kernel network stack and a zero-copy path into userspace, at the price of building reliability by hand; gRPC gives the control plane typed, evolvable and reliable calls over HTTP/2. Choosing a transport per plane, rather than forcing one everywhere, is the discipline the rest of the system follows.

Chapter 4

Protocol Buffers

The gRPC control plane of Chapter ?? rests on **Protocol Buffers** (protobuf) [?]: Google’s interface-definition language and binary wire format for structured data. It is the concrete realization of the schema-compatibility discipline this thesis returns to repeatedly — both ends must agree on a layout, and disagreement must be caught rather than tolerated (the engine’s single format definition in Section ?? is the hand-written counterpart). This chapter explains protobuf as a technology and then, in Section ??, exactly where it does and does not appear in this project — a distinction that matters, because the data plane deliberately uses a different, zero-copy cousin of protobuf rather than protobuf itself.

4.1 The schema and code generation

A protobuf schema is a `.proto` file: a set of *messages*, each a list of named, typed fields, and — decisively — each field carries an integer *field number* (its tag). The schema is the single source of truth; the compiler `protoc` generates, for each target language, the type together with its accessors and its (de)serialization code, so every participant speaks the same format without hand-written parsing. In `proto3` there are no required fields (all are optional with well-defined defaults), and the type system covers scalars (integers of several widths and signednesses, `bool`, `double`, `float`, `string`, `bytes`), `enums`, nested messages, `repeated` fields (lists), `maps`, and `oneof` (a tagged union). The field *name* is a source-level convenience; on the wire only the *number* identifies a field, which is the root of protobuf’s compactness and its evolution story.

4.2 The wire format

An encoded message is a flat concatenation of fields, each a key followed by a value. The key is a single varint packing the field number and a 3-bit *wire type*, `key = (field_number << 3) | wire_type`, and the wire type tells a reader how to consume the value even for a field it has never heard of:

wire type 0	varint	int32/64, uint32/64, bool, enum (sint via ZigZag)
wire type 1	64-bit	fixed64, sfixed64, double
wire type 2	length-delim	string, bytes, embedded messages, packed repeated
wire type 5	32-bit	fixed32, sfixed32, float

Varints store an integer seven bits per byte, the high bit marking continuation, so small numbers cost one byte; signed values use ZigZag (`sint32/64`) so that `-1, 1, -2, 2...` map to `1, 2, 3, 4...` instead of producing ten-byte negatives. A length-delimited field is a varint length then that many bytes, which is how strings, opaque `bytes`, and nested messages nest uniformly. Concretely, a message `{ function_id = 7; payload = AA BB CC }` encodes as:

```
field 1 (function_id=7): 08 07
```

```

08 = (1<<3)|0 (field 1, varint)      07 = value 7
field 4 (payload, 3 bytes): 22 03 AA BB CC
22 = (4<<3)|2 (field 4, length-delim) 03 = length  AA BB CC = bytes

```

Two properties follow directly. Fields are *order-independent and self-delimiting*: a reader keys off the field number and skips unknown fields using the wire type, so the message is not positional. And field *names never travel* — the wire carries numbers and values only — which keeps it compact but means the `.proto` is mandatory to interpret the bytes. This is the exact opposite of the data plane’s flat buffer, where fields live at *fixed offsets* with no tags and are read in place with no decode (Section ??).

4.3 Schema evolution

Because a field is identified by number and unknown fields are skippable, a schema can change without breaking peers built against an older or newer version, provided a few rules hold: a field number, once used, is never reused or repurposed; new fields take new numbers and, being optional, default cleanly when absent; and retired numbers/names are **reserved** to prevent accidental reuse. Old readers ignore (or, in many implementations, preserve) fields they do not know; new readers see defaults for fields an old writer omitted. This is precisely the “add optional fields, never reuse a tag” backward/forward compatibility that Chapter ?? credited to the control plane, and it is what lets a running cluster be upgraded component by component rather than in lockstep.

4.4 Implementing the wire format

The previous sections describe the format. This one reports what implementing it teaches. The codec behind this project’s control plane is written by hand — `cluster/src/pb.c`, about two hundred lines, no dependencies — rather than generated by `protoc`. In any other setting that would be the wrong choice: generated code is correct, complete and free.

What generation hides is that the properties attributed to Protocol Buffers in §?? follow from two decisions visible in twenty lines of a decoder.

4.4.1 Forward compatibility is one line

Every decoder in `msg.c` has the same shape:

```

while (pb_read_tag(&r, &field, &wire)) {
    if (!wire_legal(wire)) return -1;
    switch (field) {
        case 1: m->function_id = (uint32_t)pb_read_varint(&r); break;
        ...
        default: if (pb_skip(&r, wire) < 0) return -1;          break;
    }
}

```

The `default` arm is schema evolution. Not a library feature, a negotiation or a version handshake: a `switch` default that steps over what it does not recognise. A worker compiled against last month’s `.proto` decodes this month’s coordinator correctly, ignoring what it cannot name. It can step over those fields because the wire type travels in the tag, so the length of a value is computable without knowing what the value means. “Add fields freely, never reuse a tag number” is not a convention enforced by tooling; it is the condition under which that one line is sound.

The flat buffer’s counterpart is the absence of the loop. No tags, so nothing to skip, so no evolution. Two hundred lines of decoder against zero is the trade.

4.4.2 Default elision, and what it costs

A proto3 scalar equal to its default never reaches the wire, so every field writer starts with a test:

```
void pb_put_uint32(pb_writer *w, uint32_t field, uint32_t v)
{
    if (!v) return;          /* absent == zero, on purpose */
    pb_put_tag(w, field, PB_VARINT);
    pb_put_varint(w, v);
}
```

Hence a message whose fields all hold defaults encodes to zero bytes, a case the conformance vectors check. There is a real loss here: for a proto3 scalar, “absent” and “present and zero” are the same bytes and no reader can tell them apart. Proto2 had explicit presence, proto3 removed it, and later versions restored it only for **optional** and message-typed fields.

This shapes schema design directly, and the project’s schema shows it. Status codes put zero on success and every failure on a non-zero value, because a status using zero for a failure mode would be indistinguishable from an absent status. That is forced by the encoding, and it is the sort of coupling between wire format and schema that a generated codec makes easy to miss.

4.4.3 The fiddly parts

Three details account for most of the difficulty, and a naive version of each is silently wrong rather than obviously broken.

Submessage lengths precede bodies of unknown length. A length-delimited field is a varint length then that many bytes, so encoding a nested message needs its size before writing it. The usual answers are to encode twice, or to encode into scratch space and prepend. This implementation reserves a two-byte placeholder, writes the body, computes the true varint width and `memmoves` if they differ. Correct at any size, with one small move in the uncommon case. Two bytes covers bodies up to 16,383, which is every control-plane message here, so the move never happens in practice — and the vectors include a 200-byte submessage to exercise the branch that would otherwise go untested.

Varints have a maximum length, and exceeding it is undefined behaviour rather than merely wrong. A 64-bit value needs at most ten bytes ($\lceil 64/7 \rceil$). A decoder still accumulating on an eleventh shifts by seventy bits, and a shift count at least as wide as the promoted type is undefined in C.¹ So malformed input is a language-level problem, not a data problem, and the check that rejects it is required for correctness rather than defensive.

Every length off the wire is attacker-controlled. The reader bounds each one before use, and writes each comparison as a subtraction, `1 > r->len - r->pos` rather than `r->pos + 1 > r->len`. The addition can wrap and a wrapped comparison passes.² The same rule governs flat-buffer offset validation on the data-plane side, which is the more dangerous of the two: a flat buffer is dereferenced, not copied.

4.4.4 Checking the claim

An implementation that merely resembled Protocol Buffers would pass any test written against itself. So the claim is checked against an independent encoder: `cluster/test/gen_vectors.py`

¹ISO/IEC 9899:2011 §6.5.7p3.

²CERT INT30-C.

implements the same specification, in another language, without reference to the C, and `make check` compares the two byte for byte across nine messages. Two implementations derived independently from a specification agreeing tells you something about the specification. An implementation agreeing with its own test tells you it is deterministic.

The vectors include the awkward cases on purpose: an all-default message at zero bytes, a maximal ten-byte varint, negative `int64` values widened to full two's complement — the regime `sint64` and ZigZag exist for, and where choosing `int64` costs ten bytes a field — the two-byte submessage prefix, and four malformed shapes that must be rejected rather than survived. A last test appends fields with numbers the build has never seen and confirms the known ones still decode, reducing §?? to an assertion.

4.5 gRPC and Protocol Buffers together

The same `.proto` that defines the messages also defines the *service*: a set of `rpc` methods, each naming a request and response message and optionally marking either as a `stream`. `protoc` with the gRPC plugin generates the client stub and server skeleton, and gRPC carries length-prefixed protobuf messages over HTTP/2, supplying the multiplexing, flow control, deadlines, and status codes that Chapter ?? described. The four call shapes (unary, server-, client-, and bidirectional-streaming) are just which sides are marked `stream`; the wire encoding of each message is the format above.

4.6 Epilogue

Protocol Buffers identify fields by number and let a reader skip what it does not know, which is where their schema evolution comes from. Implementing the wire format by hand showed that this property rests on a single `default` branch in the decoder, that default elision shapes the schema, and that every length read off the wire must be bounded before use. The engine's data plane gives up this flexibility for a fixed layout read in place, as the next chapter explains.

Chapter 5

The Engine

In the preceding chapters technologies were discussed, that are used today to enhance distributed systems in the production stage. Kubernetes can automate scaling in cloud environments, acting like a distributed operating system, finding its best application managing multiple cpus across various machines, but as it's evident it's best applied when used on specific hardware, with intense localization, in which case communication costs drop by orders of magnitudes and provide a great setting for the distributed architectures in use today. What became also evident was that the communication costs usually far outweigh the benefits of having an extra cpu or multiple of them across the room or the city/world, making the central design and the area or application the most important factors, with benefits mostly measured with latency, in cases where small jobs are dispatched, not so much when there is great concurrency. As a result, despite the higher development time it certainly pays to incorporate gRPC or a protocol buffer architecture in both microservices setups or more scientific/compute focused applications, hence the relatively high adoption rate today. Finally, the epitome of speed on computation, is the XDP, here the eBPF suite is able to provide a seemingly magic solution, albeit subject to a number of limitations however, and it's mostly used as a logging aggregation tool. The discussion on execution and system calls in linux, brings us to the CRIU project, which is not best suited for relocation of isolated processes, and in a healthy cluster would not make sense to relocate a running container via that, it could whoever find applications on a similar project.

The engine is deliberately a *subset*. It implements job definition, identity, placement, dependency tracking, code shipping, and the feed of one job's output into another's input. It does not implement failure recovery, concurrent dispatch, or multi-head routing. Section ?? states each omission and where it would attach, because an honest account of what a prototype does not do is more useful than a diagram of what it might.

5.1 The high level design

The objective of this work is to design a system that's capable of utilizing two or more computers in order to perform a certain task or an arbitrary number of them faster than one of the machines would be, but not in an embarrassingly parallel or trivial way. What I have chosen to design is a somewhat loosely coupled system where the cost of communications are still prevalent and extremely important, but are baked in the system in a way that is scalable, and configurable. For example the kubernetes management of the lifecycle could provide the basis for dynamically adding nodes to the network, and spawning worker nodes there. Also since the aim is not to just fan out or map reduce, but achieve actual results based on non trivial dependencies of the jobs executed we opted for a design that lets one submit for computation a series of codependent jobs. Actually, codependent is not the right word here, since what we define is a DAG: directional acyclic graph that defines dependencies of jobs, defining which jobs need the output of others to start processing. One or more jobs may be used as input to the "sink". Also with the use of

thread pools on the workers we can actually implement rather efficiently a somewhat trivial fan out technique, that's configurable i.e by setting a header flag as parallelization and in so doing being able to break down the fed data into multiple sub copies.

5.2 The job model

The definition of the work "unit" in the system is the *job*, it starts its life as a pair of data and a function identifiers. The function is a loadable .so static object file that can be loaded dynamically, the .sos are going to be distributed along the working nodes and not retransmitted via the wire on runtime, since most systems can be designed with the expectations of the loads that they are going to face in mind. Also large jobs are going to be transmitted via TCP for payload integrity, since a lost data piece would halt a whole pipeline and perhaps destroy resulting data making them unusable. What is being implemented is a naming scheme with a tag for a data set and each pre-distributed function, a pair of which constitutes a job and which is also assigned an id, these ids could be hashes so that they are easily calculated and hashes could easily be used to help implement result storage, say for example via a distributed hash table. The goal is to have a configurable yet easily adaptable to additions or exemptions of nodes for result storing or computation when needed, and most importantly for a distributed system, this would allow the system to function without having to distribute the information to each node: if the result is easily traceable then not everyone needs to know about all the parts of a graph, and where they are being executed, and if someone needs to read one they can easily get the job id, hash it and calculate the node on which it's been saved. Of course the worker could hold no catalog of functions and it would be beneficial for it to not need no prior knowledge of the work it is about to do; everything it must know arrives in the bytes it has just read, a short lived job could no longer burden the system if it's been assigned a 'no cache' flag.

In most cases the unit of data is a *dataset*: rows of fixed-width integer vectors, each row carrying its own identifier. The unit of code is a compiled shared object, or more promptly, it's an id for the .so that's been assigned to it via a hash, so that it's consistent, and we can have it transmitted via a leader after start up of either the vms, the machines or the containers that are part of the 'cluster'. A *jobGraph* is a set of jobs together with their dependencies, and a job's input is either a file or the concatenated rows of its dependencies' outputs. The graph is defined as a set of jobs, and the jobs having as intake the output of another have as input the id of the job. The DAG is constructed and then reversed, this way we know which jobs need to take place first. After this has happened in general a load balancer could make the decision to either assign the job to the local machine or dispatch it towards the network. The load balancer could be L7 application via Kubernetes, or L4 via kubernetes or L4 via the actual connections the heads are managing. Of course, it could also be an issue of storing the result, one could argue that the result is best saved on an appropriately chosen machine, for example a hash of the job modN could give us the id of the worker node, that ll store the results, with the caveat that the network can be easily saturated. In fact, for a simple internet connection one could have up to a 1MB/s of upload, and jobGraphs should output a significant amount of data if they are to benefit from this model of execution, which in this DAG-approach is a synchronized (weakly) pipeline generator.

```
worker 127.0.0.1:16001
worker 127.0.0.1:16002

job fibA so=funcs/fib.so fn=fib data=input/vectors_a.bin flags=parallelizable
job fibB so=funcs/fib.so fn=fib data=input/vectors_b.bin flags=parallelizable
job total so=funcs/sum.so fn=sum deps=fibA,fibB
```

Two producers compute Fibonacci numbers over independent inputs; a third consumes both. That last line is the whole point of the exercise — it is the edge across which one job's result

becomes another job’s input, on a different machine, without either job knowing the other exists.

Two programs implement this. The **head** reads the graph, assigns identities, decides placement, tracks dependencies, and dispatches. The **worker** receives one task, loads the function it carries, runs it over the data it carries, and replies. There is no third component: no coordinator, no metadata service, no queue.

5.3 The object model

Every object begins with the same twenty-four byte envelope:

offset	size	field
0	4	magic
4	2	version
6	2	type
8	8	objectId
16	8	totalLength

and there are four types.

- **DATA** — 56 byte header plus an opaque payload. Bytes with an identity. A *result* is a DATA and nothing else, which is what allows one job’s output to be another job’s input with no conversion anywhere.
- **FUNCTION** — 112 byte header plus the compiled shared object. The header carries the ABI version and a fixed 64 byte symbol field, so the code always begins at a constant offset.
- **JOB** — 96 bytes, fixed, carrying nothing. The function, the input and the output are named by identifier. Also the parent job, the part index and count for a divided job, the state, and the source and destination nodes.
- **REQUEST** — 56 bytes, fixed. “Send me the object of this type with this identifier.”

Three consequences follow from this layout, and they are the reason for it.

Framing is free. `totalLength` sits at offset 16, inside the first twenty-four bytes. A reader takes the envelope, learns the length, reads the remainder, and the next object begins immediately after. There are no delimiters, no escaping, and therefore no byte sequence that a payload must not contain. The same bytes are valid on a socket, in a pipe, and in a file, so a job written to a file today is byte-identical to the job that would have arrived over a connection — which is what lets a single reader implementation serve all three.

A job is 96 bytes. Moving work between machines costs less than a network packet. Moving its *data* may cost megabytes, and the two decisions are now separable. This is the single most important property for a system whose links are ordinary Ethernet: the correct instinct becomes *move the job to the data*, and the format permits it.

Everything is addressable. DATA, FUNCTION and JOB all have an identifier at the same offset, so a generic reader can identify, index and skip any of them without knowing which it is holding. REQUEST exists because that addressability is only useful if a node can ask.

There is deliberately no response type. A node that holds what was requested sends the object back as it stands — its header already states the identifier and the length, so nothing needs wrapping or restating — and a node that does not sends the same REQUEST back with

its status field set. The asker distinguishes the two the way it distinguishes any two objects: by reading the type it was handed. There is no success status, because success is the object arriving. Furthermore, the format of the messages of the nodes is now built to be simple, rather than exact and a finished product.

5.4 One wire format

The experimental groundwork for this engine was four client/server pairs, each exploring a different execution strategy, and each of which grew its own frame layout — five wire formats and three incompatible function signatures across six projects. The engine’s first design decision is therefore a negative one: there is exactly one definition of the format, and the head, the worker and every job function compile against it.

```
task frame      head -> worker
[u32 totalLength][ 48-byte header | symbol | data | code ]

    0 magic 0xE9C1FEED      20 nameOffset      24 nameLength
    4 version      28 dataOffset      32 dataLength
    8 jobId (u64)      36 codeOffset      40 codeLength
   16 flags      44 reserved

reply frame     worker -> head
[u32 status][u32 outLength][outLength bytes]

dataset
[u32 magic][u32 version][u64 rowCount][u32 cols][u32 reserved]
rowCount x [u64 rowId][int64 v0 .. v(cols-1)]
```

Offsets rather than a packed struct: the frame is self-describing, so the regions can be validated independently and a stale sender fails on magic or version rather than by silently misreading a field. The reply carries a **status**, which is a small thing that matters — a failed job is *reported*, where the earlier prototypes signaled failure by dropping the connection and left the caller to infer the cause.

One class of bug in those prototypes is worth recording, because it appeared independently in every one of them. Each validated an attacker-controlled row count with arithmetic that overflowed before the check could reject it: a count near 2^{61} makes $8 + count \times 8$ wrap to a small value, the guard passes, a small buffer is allocated, and the loop then writes past it. In the data gram-based pairs this was reachable from a single unauthenticated packet. The engine’s **dsBytes** bounds the count *before* multiplying, which is the only ordering that is safe, and the constant that bounds it (**DS_MAX_ROWS**) is part of the format rather than a local convention.

The format that the engine has implemented is that of an ABI, that is a `contract.h` that specifies the signature of the payloads: and the objects that can be referenced, represented, communicated and transmitted/shared over the network.

5.5 Identity: ids computed from the bytes

Every identifier is a hash of the bytes it names, using FNV-1a over 64 bits:

- a **row id** is the hash of that row’s value bytes, so a row keeps its identity no matter which job produced it or how many merges it has survived;
- a **job id** is the hash of the function name, then the code, then the input, so the same function over the same input is the same job — on any head, in any process, forever.

This is content addressing, and it is cheap to adopt at this scale while paying for itself several times over. It gives deduplication for free: two submissions of identical work collide by construction. It gives idempotence, because a repeated dispatch is recognizably the same

dispatch. It gives a cache key for the function artifact, which is the mechanism by which the shared object stops traveling with every task. And it gives determinism that can be checked from outside the system: the row identifiers produced by the engine were reproduced by an independent implementation written against the format specification alone, which is a stronger statement about the format than any amount of internal testing.

This is something that could be implemented either via a redis store, or a postgres store procedure (especially since we know of the ways that the results or general content payloads are going to be in the network). The road we have taken is that of using the etcd, a fundamental component of kubernetes, the only persistent layer and it's something we briefly describe in the previous chapter.

Inputs are content-addressed and job functions are pure, so any lost result can be recomputed by replaying its subgraph. The ingredients for lineage-based recovery — the model Spark uses in place of replicating intermediate data — are already present; only the bookkeeping is missing, but if the 'leader' or outwards facing head node can store the This is noted again in Section ??.

5.6 Placement by reverse traversal

The head places jobs by walking the graph *backwards*. Sinks — jobs nothing depends on — are assigned first; each producer is then placed relative to the consumer that was placed before it.

The ordering is what makes this worth doing. Once a consumer has a node, every producer above it knows where its output must go *before it runs*. Results can therefore be pushed forward to a destination that is already known, rather than published somewhere and then discovered by a lookup. A design that places producers first has no such luxury: it must either buffer results centrally or introduce a directory service to answer the question that reverse traversal answers for free. This is describing however only one possible implementation to solve the issue. The main problem is that a zig zag across machines would be stupidly expensive if it comes with carrying along data that are large enough to make parallel execution worthwhile i.e. huge data sets and complex functions, since as we have seen many times over the communication costs are prohibited especially when talking about internet connections, and initializing of connections and data transfer, so unless the workload is extremely large, a 'local' - greedy balancer would make things perhaps worse if left unchecked.

A strong bias for cpu affinity would come into play especially if all head nodes were to receive job sets, resulting to dispatching in small cases and in so doing basically parallelizing most of the stuff.

For a start however one could try this free standing policy: a producer inherits its consumer's node by default, which keeps the feed node-local; a producer marked `parallelizable` in the graph is spread to the next node instead. With the three-job graph above:

workers	fibA	fibB	total	
1	w1	w1	w1	
2	w2	w1	w1	
3	w2	w3	w1	
4	w2	w3	w1	(three jobs; w4 idle)

The flag is the only scheduling hint the graph carries, and it is enough to express the distinction that matters here: work that must be near its consumer, and work that should be spread away from it. The main problem solved is a lack of need of communication in cases where results are ready in one or two nodes and one other node needs to find them and collect them to start processing, something that could however be solved by knowing the jobs hash and sending the bytes that way, with the speculation that the job is there, is ready to run, and that it wont bottleneck execution on multiple fronts or is not going to deadlock multiple -not threads- but machines.

5.7 Dependency tracking without consensus

Each job holds a counter of unsatisfied dependencies. When a job completes, the counter of every job depending on it is decremented; a job whose counter reaches zero is ready to dispatch. This is the standard dependency-counter formulation, and the standard treatment of it spends considerable effort on the race: two producers finishing simultaneously, both reading the same counter, both writing the decremented value, and the join silently never firing. The usual remedies are an atomic decrement in a transactional store, or a coordination service holding the counter on everyone's behalf.

The engine needs neither, and the reason is structural rather than clever. Placement is deterministic and the graph is known, so each job has exactly one owning head, and only that head ever writes that job's counter. A single writer cannot race with itself. The concurrency problem was not solved; it was *removed*, by an ownership rule that placement already had to establish for other reasons.

This property is worth stating precisely because it is the part that does not survive carelessly. It holds as long as ownership is unambiguous — which means it depends on every head computing the same placement, which in turn depends on every head seeing the same membership. Membership change, not failure, is the first thing that will break it, and it is the reason the design calls for a membership epoch stamped onto each graph at submission.

5.8 Shipping code

The worker receives the function as bytes and must run it without the artifact ever existing as a file. It creates an anonymous in-memory file with `memfd_create`, writes the received object into it, and calls `dlopen` on `/proc/self/fd/N`. Nothing touches disk, and because each task gets a fresh inode, the dynamic linker's path-based cache cannot return a previously loaded object that happened to share a name — a failure mode that is invisible until two jobs disagree about what a symbol means.

This mechanism was developed in the first of the four prototype pairs and is reused unchanged. The prototypes established a ranking between the three ways a worker can obtain code, fastest first: a resident function compiled into the worker; flat position-independent machine code copied into a mapping made executable with `mprotect`; and a shared object loaded through the dynamic linker. The engine uses the slowest of the three deliberately. The ranking measures the *loading* step, which content addressing is about to make rare, and in exchange the shared object brings a symbol table, a defined ABI, and ordinary tooling — a flat blob has none of these, and its ABI in the freestanding prototype omitted the output capacity argument, which is precisely how it came to write past its buffer.

5.9 Two heads: C and Java

The head was built twice, in C and in Java, over an identical worker, an identical format, and identical job functions. The only variable is the head's language; the build refuses to proceed if the shared files have drifted, since at that point the comparison means nothing.

The port is not free, and the cost is exactly one file. In the C engine the format has a single definition that all three parties compile against, so drift is impossible by construction. A Java head cannot participate in that: it needs its own transcription of the frame layout, maintained by hand, with nothing in either language able to detect the moment the two stop agreeing. Three specific hazards appear inside that transcription, and all three produce plausible-looking wrong answers rather than errors: Java's `ByteBuffer` defaults to big-endian where the C side writes native little-endian; `DataStream.readInt` is likewise big-endian and cannot be used

to read the reply header; and Java’s signed bytes must be masked before entering the hash, or the identifiers diverge from the C implementation’s for any input containing a byte above 127.

What the port buys is equally concrete. The dataset, a family of loose functions over a `void *` in C, becomes an object that knows its own shape, and merging several datasets becomes four lines. Graph parsing and the placement walk are similarly shorter and harder to get wrong.

The conclusion matches the split this thesis has maintained throughout, and the line falls where the bytes are. Java is the better language for the graph, the placement policy, the parsing, and — the argument that carries the most weight in a Kubernetes deployment — for watching the API server, which is substantial work in C and nearly free with a client library. C is the better language wherever payload is touched, and it remains the only reasonable choice for the worker. The productive arrangement is therefore not to choose, but to split the head: control in Java, byte movement in C, with the format defined once on the C side. A head that stays off the data path can be written in either language; a head that carries results cannot be written in Java without paying for it.

Also the hexagonal architecture used here with the adapter, domain, port and service packages can also further simplify the design and development of such a project, which in C are typically harder to onboard. Further the controller logic should be a fraction of the total compute time, even in a generally more performant language like C.

5.10 Deployment

The engine runs as a set of containers, one per node, on a user-defined network so that nodes address each other by name rather than by address. A single image serves every role — build environment, worker, and head — and the project directory is bind-mounted rather than copied in, so that rebuilding a binary does not mean rebuilding an image. Workers therefore all listen on the same port, distinguished by hostname.

The nodes are provisioned as debugging environments rather than as minimal runtime images: each carries `gdb`, `strace`, `tcpdump` and the traffic-control tools, and runs with the capabilities those require. This makes the properties discussed above directly observable — the frame header can be read off the wire between two containers, the `memfd_create` and `dlopen` sequence can be traced as a task arrives, and artificial latency or packet loss can be introduced on a single node’s interface to see how the engine behaves when the network stops being ideal.

Kubernetes is intentionally absent from this iteration. Everything Chapter ?? describes applies — a worker fleet is a Deployment, the head is a StatefulSet with stable per-pod DNS, and membership comes from the API server — but the engine’s own mechanisms are easier to observe and to falsify when the orchestrator is not also in the picture. The step from named containers on a network to named pods on a cluster is small, and it is the natural next one.

5.11 What the engine demonstrates, and what it does not

Running the three-job graph across two worker containers produces the expected placement, the expected seven result rows, and identical output from the C head and the Java head. Two invariants are worth separating, because they fail differently. Row identifiers depend only on row values and must match across heads, compilers and machines. Job identifiers additionally hash the function’s compiled bytes, so they are stable only for a fixed toolchain — rebuilding the functions with a different compiler moves every job id while leaving every row id untouched. This is the correct behaviour for a content-addressed system and an easy result to misread as a bug.

The omissions are as follows.

- **Failure handling.** A worker that cannot be reached fails the whole graph. There is no

retry, no re-placement, no timeout. As Section ?? notes, content-addressed inputs over pure functions already permit recovery by recomputation; what is missing is the bookkeeping to know what to recompute, not the ability to do it.

- **Concurrent dispatch.** Independent jobs placed on different nodes are dispatched one after another. Placement is therefore demonstrated but not yet exploited, and no speedup should be claimed from the current implementation.
- **A single head.** One head owns every job in the graph, which is what makes the single-writer argument of Section ?? trivially true. The claim becomes interesting, and requires the membership epoch, only with several heads.
- **Function caching.** The shared object is re-sent with every task, although the job id already contains the hash that would make a per-node cache correct. This is the shortest path from the current prototype to a meaningful reduction in bytes on the wire.
- **Transport selection.** One transport is used for everything. The size-based split between a bulk path and a low-latency path, and the XDP work of Chapter ??, remain design rather than implementation.

What the engine does establish is the part that is hardest to retrofit: a single authoritative wire format, identity derived from content rather than assigned by a counter, a placement rule that makes destinations known before results exist, and an ownership rule that eliminates the coordination problem instead of solving it. Each of the omissions above attaches to that skeleton without disturbing it, which was the objective.

5.12 Technologies considered and not used

Several of the decisions above have well-established alternatives. Each is worth stating with the reason it was not taken, since a design is as much what it refuses as what it contains.

5.12.1 Raft, implemented rather than consumed

The system depends on distributed consensus and does not implement it. Raft [?] was designed explicitly for understandability relative to Paxos, and an implementation is a realistic undertaking: leader election, log replication, membership change and snapshotting.

It was not implemented, for three reasons.

First, the system needs consensus for a very small amount of state — node presence and holdings — and needs it to be *correct* rather than fast. A hand-written Raft would be the least tested component in the project while being the one whose failure is hardest to observe, since a split-brain does not crash anything, it merely produces two plausible answers.

Second, the properties actually relied upon are the ones an implementation gets wrong last: leases that expire without a coordinator, linearizable reads, and correct behaviour during a leader change. `etcd` provides these and is exercised continuously as the backing store of Kubernetes, discussed in Chapter ??.

Third, and most importantly, consensus is deliberately *off* the data path here. Nothing about a job, a payload or a result passes through it. Writing a Raft implementation to carry two keys per node per second would be a large investment in the part of the system that does the least work.

It is worth noting what an in-house implementation would have bought: control over the lease semantics, the ability to co-locate the store with the node and remove a process, and the removal of an external dependency from the deployment. For a thesis whose subject is the underlying mechanisms, that is not a negligible argument — it is simply outweighed by the risk of an unobservable correctness bug in the component everything else trusts.

5.12.2 ZooKeeper and Consul

Both solve the same problem. ZooKeeper’s ephemeral znodes are very close to the lease mechanism used here, and its watch primitive would remove the polling that the current design performs. It was not chosen mainly for operational weight: a JVM process with its own configuration and failure modes, in a project that already has a JVM component with different concerns. Consul adds service discovery and health checking that the system does not need, since a node’s presence claim already is its health check.

5.12.3 Gossip instead of consensus

A membership protocol such as SWIM [?] would remove the external store entirely. Nodes would exchange load and holdings directly, with failure detection by suspicion and dissemination by piggyback. It scales considerably better than a consensus store — no single quorum, no leader, load spread across participants — and for load and holdings, which are advisory and tolerate being seconds stale, eventual consistency is entirely sufficient.

This is the most defensible alternative and would be the first thing to try at a larger node count. It was not chosen because at two nodes it is strictly more code for strictly less certainty, and because the cluster state is currently read far more often than it is written.

5.12.4 A distributed hash table

For locating results, a DHT in the style of Chord [?] would replace the holdings map. Every node computes the same answer for “who should hold this identifier” from the identifier alone, and no directory is needed at all — a natural fit for a system that already content-addresses everything.

The system leans towards this without adopting it. What is implemented computes ownership as a hash modulo the node count, which is correct but reshuffles almost every key when a node joins. Rendezvous hashing would move only $1/N$ of the keys and is the obvious improvement. The full DHT with routing and replication is unnecessary at this scale, where any node can hold the addresses of all others.

5.12.5 A message broker

Kafka or RabbitMQ between the heads would provide durability, replay, consumer groups and back-pressure — the last of which the current design genuinely lacks. A job queue is, after all, a queue.

It was rejected because it inverts the property the system is built on. A broker is a store-and-forward intermediary: work is written to it and read from it, which means every job and, if payloads were included, every byte of data would traverse a third machine. The entire argument of Section ?? is that the job should travel to where the data already is. A broker in the middle makes that impossible by construction, and it is the same objection that rules out putting jobs into `etcd`.

5.12.6 gRPC, and other serialisation formats

Chapters ?? and ?? discuss gRPC and Protocol Buffers in detail, and a reasonable reader will ask why neither is used here. The answer is specific rather than dismissive.

Protocol Buffers are a *schema* format: fields are tagged, varint encoded, and the reader reconstructs a message into language objects. That is exactly right for evolving service interfaces and exactly wrong for this system’s hot path, where a payload is an opaque block of bytes that must reach a C function’s address space without being copied or reinterpreted. The format here

is a fixed binary layout with every offset a compile-time constant precisely so that a worker can read a payload straight off a socket into an arena and hand the function a pointer.

The price of that decision is real and should be stated: no schema evolution beyond a version field and reserved bytes, no generated code, no cross-language tooling, and a second hand-maintained copy of the format for the Java side — mitigated, but not eliminated, by generating it.

Cap'n Proto or FlatBuffers would have been the closer comparison, since both allow reading fields in place without a parse step, and both would have supplied schema evolution that the current format lacks. They were not used mainly because the format here is small enough to define completely in one header that the C side compiles against directly, which keeps a single authority over the layout.

5.12.7 MPI and the HPC tradition

MPI is probably the default when it comes to having to speed up computation, by extending the program on a cluster of computers, however what this gives is the possibility of being able to run a generic program written in C, if one takes the time to rebuild it in a specific way that's a bit more agnostic to the data it reads in and the way that the artifacts are being instrumented to work together.

5.12.8 Kubernetes as the scheduler

Chapter ?? covers Kubernetes, and it is capable of placing work across machines. Its unit is a container, its scheduling latency is measured in seconds, and its model assumes long-lived services rather than short jobs. Using it to place a 96 byte job would be a category error. Its natural role here is the one it already has in the deployment: managing the lifecycle of the nodes themselves, inside which this engine schedules jobs at millisecond granularity.

5.12.9 A shared filesystem or object store

NFS, Ceph or S3 for the datasets would remove the transfer problem: every node sees every artifact. This trades a network transfer this system controls for one it does not, and it gives up the property that a node knows locally whether it can run a job. It also concentrates load precisely where the system tries to avoid it. For a cluster with an existing distributed filesystem the trade may well be worth it, and the catalogue port is the seam at which such a backend would attach.

5.12.10 Faster transports

RDMA would eliminate the copies in the transfer path; DPDK or AF_XDP — the latter discussed in Section ?? — would bypass the kernel network stack. All three are appropriate when the transport itself is the bottleneck. On the hardware this system targets, an ordinary 1 Gb or 2.5 Gb link, the limit is the link and not the stack: 2.5 Gb is roughly 312 MB/s, and a single TCP stream saturates it using about one core with standard offloads enabled. The measures that matter at that scale are avoiding transfers altogether, batching small objects so that a 96 byte job does not occupy its own packet, and compressing payloads — LZ4 compresses at roughly an order of magnitude faster than the link can carry the result, so it is free headroom.

5.13 Epilogue

The engine implements job definition, content-derived identity, placement by reverse traversal, dependency tracking without consensus, code shipping through `memfd_create` and `dlopen`, and

the feeding of one job's output into another's input. It was built twice, with a C head and a Java head, over one wire format. Failure recovery, concurrent dispatch and multiple heads are deliberately left out; the conclusions that follow state these omissions and where each would attach.

Chapter 6

Conclusions and suggestions for improvement

This thesis set out to combine two or more ordinary computers into one of larger capacity without reducing the problem to an embarrassingly parallel one. The engine of Chapter ?? answers the question at the level of structure: a single authoritative wire format, identity derived from content, placement that makes destinations known before results exist, and an ownership rule that removes the coordination problem instead of solving it. What it does not yet do, and where each missing piece would attach, is set out below.

6.1 Limitations and what attaches next

In the spirit of Section ??, the omissions are stated plainly.

- **Shipping duplicates payloads.** Sending a job together with its function and data means the payload then exists on both machines. Sending only the 96 byte job requires the receiver to be able to ask for what it lacks, which is the same missing capability as the first item.
- **No back-pressure.** The pending queue is unbounded and the pool drops a task when every deque is full. One fast node can bury a slow one. The refusal status already exists in the protocol as the shape of the answer.
- **Division is unsound for the current functions.** The splitting machinery cuts a payload at byte boundaries, which is meaningless to a function that reads a header first — which all of the supplied functions do. Either the head understands the payload format, reintroducing the coupling that opaque payloads removed, or the shared object supplies its own splitter and merger alongside its job function. The second is consistent with the rest of the design: the code that understands the bytes is already shipped with them.
- **Security is absent by construction.** The worker `dlopen`s bytes that arrived over a socket. That is remote code execution as a feature, with no authentication anywhere in the protocol. On a unix socket the filesystem permissions are the entire security model. The moment the worker accepts objects over TCP, that ceases to be true, and an authenticated handshake becomes a prerequisite rather than an enhancement.

What the system does establish is narrower than a production scheduler and, I would argue, more durable: a wire format in which framing, identity and addressing are properties of the bytes rather than of the protocol around them; an execution model in which work is idempotent by construction, so that recovery is recomputation and needs no transactions; a coordination layer that carries decisions rather than data, and in which membership expires by itself; and a

demonstrated transfer of work between two independent machines that shared nothing but the definition of the format.

Each of the omissions above attaches to that structure without disturbing it, which — as in the previous chapter — was the objective.